

# Python进阶语法-数据容器

数据容器是Python中用来存储和组织数据的结构，常见的包括列表、元组、字符串、集合和字典

## 1. 列表

列表是一种有序的可变数据类型，用于存储多个元素。列表中的元素可以是不同类型的数据。

列表三个特点：有序、元素可变、元素可重复

### 1.1. 列表的定义方法

定义列表的语法如下：

#### ▼ 列表定义示例

Python |

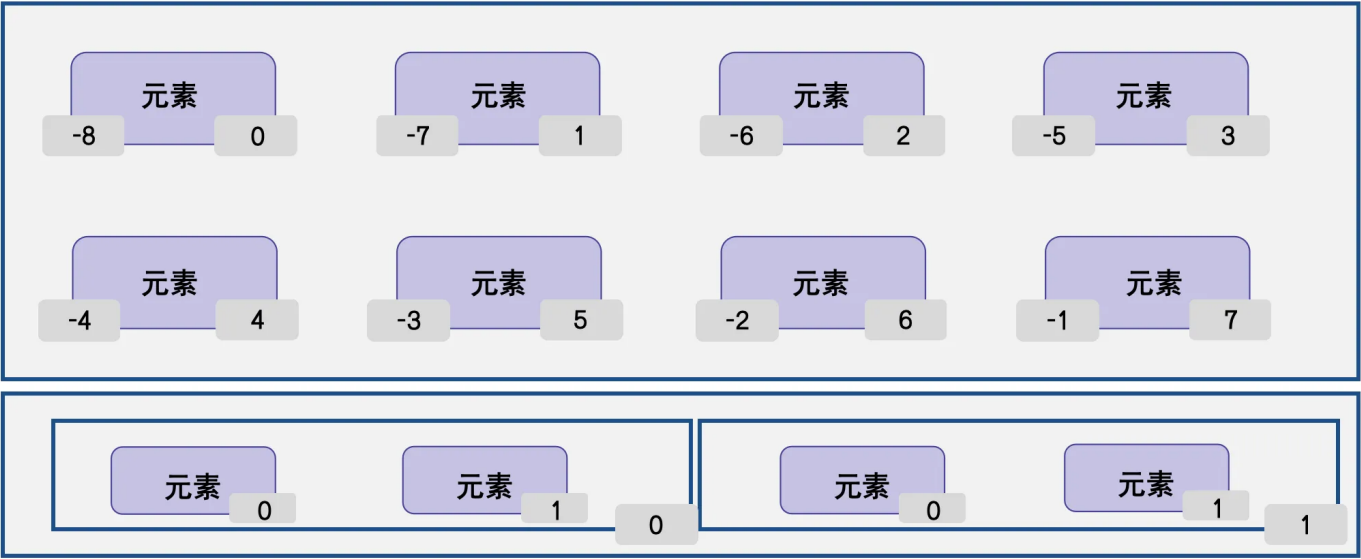
```
1  # 创建一个空列表
2  my_list = []
3
4  # 创建一个包含多个元素的列表
5  my_list = [1, 2, 3, 4, 5]
6
7  # 列表可以包含不同类型的元素
8  mixed_list = [1, "hello", 3.14, True]
```

### 1.2. 列表的下标索引

列表中的元素通过下标（索引）进行访问。下标从 0 开始，下图中的蓝色框代表列表，分别有两个列表

第一个列表中有8个元素，第二个列表中有2个元素（两个元素都是列表，并且每个列表中各有2个元素）

图中标示了这些列表的下标索引



下标索引访问示例代码：

▼ 列表下标索引访问与修改示例 Python |

```
1 my_list = [10, 20, 30, 40, 50]
2
3 # 访问第一个元素
4 print(my_list[0]) # 输出: 10
5
6 # 访问最后一个元素
7 print(my_list[-1]) # 输出: 50
8
9 # 修改某个元素
10 my_list[1] = 25
11 print(my_list) # 输出: [10, 25, 30, 40, 50]
```

1.3. 列表的常用操作总结

| 操作          | 方法/属性                        | 示例代码                               |
|-------------|------------------------------|------------------------------------|
| 查找某元素下标     | <code>.index(元素)</code>      | <code>my_list.index(20)</code>     |
| 统计某元素数量     | <code>.count(元素)</code>      | <code>my_list.count(10)</code>     |
| 数组中元素个数     | <code>len(列表)</code>         | <code>len(my_list)</code>          |
| 在指定下标位置添加元素 | <code>.insert(下标, 元素)</code> | <code>my_list.insert(2, 35)</code> |
| 追加元素        | <code>.append(元素)</code>     | <code>my_list.append(60)</code>    |

|                 |                             |                                       |
|-----------------|-----------------------------|---------------------------------------|
| 追加数组            | <code>.extend(另一个列表)</code> | <code>my_list.extend([70, 80])</code> |
| 删除指定元素（匹配到的第一个） | <code>.remove(元素)</code>    | <code>my_list.remove(30)</code>       |
| 删除指定下标的元素并返回该元素 | <code>.pop(下标)</code>       | <code>my_list.pop(1)</code>           |
| 删除指定下标的元素       | <code>del 列表[下标]</code>     | <code>del my_list[2]</code>           |
| 清除列表中的所有元素      | <code>.clear()</code>       | <code>my_list.clear()</code>          |
| 列表拼接            | 使用 <code>+</code> 操作符       | <code>my_list + [60, 70]</code>       |
| 检查元素是否在列表中      | 使用 <code>in</code> 关键字      | <code>20 in my_list</code>            |

## 1.4. 列表的遍历（循环）

遍历列表是处理列表中元素的一种常见操作，可以使用 while 循环或 for 循环来实现

### 1.4.1. 使用 while 循环遍历列表

while 循环根据条件重复执行代码块，常用于需要手动控制循环变量的情况

▼

列表While循环遍历示例

Python |

```

1  my_list = [10, 20, 30, 40, 50]
2  i = 0
3
4  while i < len(my_list):
5      print(my_list[i])
6      i += 1
7
8  # 上述代码中，我们使用 while 循环遍历列表 my_list，并打印每个元素
9  # 循环变量 i 从 0 开始，每次循环后递增 1，直到 i 等于列表的长度

```

### 1.4.2. 使用 for 循环遍历列表

for 循环更加简洁，适用于需要遍历整个列表的情况

```
1 my_list = [10, 20, 30, 40, 50]
2
3 for element in my_list:
4     print(element)
5
6 # 上述代码中，我们使用 for 循环遍历列表 my_list，并打印每个元素
7 # for 循环会自动处理循环变量的增加
```

当然也可以使用 for 循环和 range 函数来遍历列表的索引，如下所示

```
1 my_list = [10, 20, 30, 40, 50]
2
3 for i in range(len(my_list)):
4     print(f"Index {i}: {my_list[i]}")
5
6 # 上述代码中，range(len(my_list)) 生成从 0 到列表长度减 1 的序列
7 # for 循环遍历这个序列并打印每个索引和对应的元素
```

## 2. 元组

元组 (tuple) 是 Python 中的一种数据结构，与列表 (list) 类似，用于存储多个项目。元组是不可变的，这意味着一旦创建，它的元素就不能被修改、添加或删除

元组三个特点：有序、元素不可变、元素可重复

### 2.1. 元组的定义方式

定义元组的语法如下：

### 元组定义示例

Python |

```
1  # 创建一个空元组
2  empty_tuple = ()
3
4  # 创建一个包含多个元素的元组
5  my_tuple = (1, 2, 3, 4, 5)
6
7  # 元组可以包含不同类型的元素
8  mixed_tuple = (1, "hello", 3.14, True)
9
10 # 不使用括号也可以定义元组（逗号分隔）
11 another_tuple = 1, 2, 3
12
13 # 创建一个只包含一个元素的元组（需要在元素后加逗号）
14 single_element_tuple = (1,)
```

和列表一样，元组同样可以通过下标索引访问

### 元组访问元素示例

Python |

```
1  # 定义元组
2  my_tuple = (10, 20, 30, 40, 50)
3  print(my_tuple) # 输出: (10, 20, 30, 40, 50)
4
5  # 访问元组中的元素
6  print(my_tuple[0]) # 输出: 10
7  print(my_tuple[-1]) # 输出: 50
```

由于元组是不可变的，一旦创建，不能修改其元素。如果尝试修改元组的元素，会引发 `TypeError`

r

### 元组修改元素示例

Python |

```
1  my_tuple = (10, 20, 30)
2
3  # 尝试修改元组中的元素
4  try:
5      my_tuple[1] = 25
6  except TypeError as e:
7      print(e) # 输出: 'tuple' object does not support item assignment
```

## 2.2. 元组的常用操作总结

| 操作         | 方法/属性                   | 示例代码                             |
|------------|-------------------------|----------------------------------|
| 访问元组中的元素   | 使用下标                    | <code>my_tuple[1]</code>         |
| 查找某元素下标    | <code>.index(元素)</code> | <code>my_tuple.index(20)</code>  |
| 统计某元素数量    | <code>.count(元素)</code> | <code>my_tuple.count(10)</code>  |
| 获取元组长度     | <code>len(元组)</code>    | <code>len(my_tuple)</code>       |
| 元组拼接       | 使用 <code>+</code> 操作符   | <code>my_tuple + (60, 70)</code> |
| 检查元素是否在元组中 | 使用 <code>in</code> 关键字  | <code>20 in my_tuple</code>      |

## 2.3. 元组的解包操作

元组的解包（unpacking）操作可以将元组中的元素赋值给多个变量。这样可以方便地从元组中提取多个值

### 元组解包示例

Python

```

1  # 元组解包
2  my_tuple = (10, 20, 30)
3  a, b, c = my_tuple
4
5  print(a)  # 输出: 10
6  print(b)  # 输出: 20
7  print(c)  # 输出: 30
8
9  # 只解包部分元素
10 my_tuple = (10, 20, 30, 40, 50)
11 a, *b, c = my_tuple
12
13 print(a)  # 输出: 10
14 print(b)  # 输出: [20, 30, 40]
15 print(c)  # 输出: 50

```

## 3. 字符串

字符串本质上就是字符的数据容器，和列表、元组一同属于序列类型

字符串同样有下标索引，但字符串与元组一样是不可修改的，这意味着如果要修改字符串，必须创建一个新的字符串

字符串三个特点：有序、元素不可变、元素可重复

### 3.1. 字符串的常用操作总结

| 操作        | 方法/属性                           | 示例代码                                     |
|-----------|---------------------------------|------------------------------------------|
| 查找某元素下标   | <code>.index(字符)</code>         | <code>my_string.index('a')</code>        |
| 统计某元素数量   | <code>.count(字符)</code>         | <code>my_string.count('a')</code>        |
| 查看字符串长度   | <code>len(字符串)</code>           | <code>len(my_string)</code>              |
| 字符串中字符的替换 | <code>.replace(旧字符, 新字符)</code> | <code>my_string.replace('a', 'b')</code> |
| 字符串的分割    | <code>.split(分隔符)</code>        | <code>my_string.split(',')</code>        |
| 字符串的规整    | <code>.strip()</code>           | <code>my_string.strip()</code>           |

示例代码：

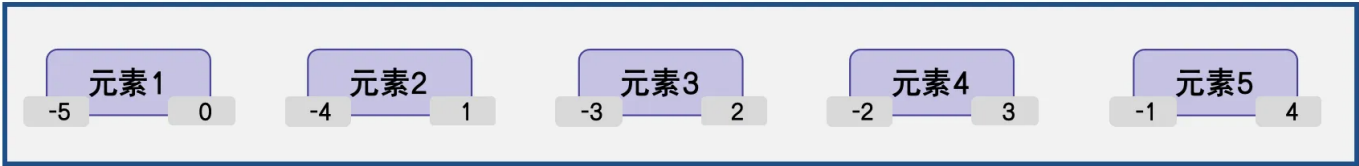
#### ▼ 字符串常用操作示例

Python |

```
1 my_string = " hello, world! "  
2  
3 # 查找某元素下标  
4 print(my_string.index('o')) # 输出: 5  
5  
6 # 统计某元素数量  
7 print(my_string.count('l')) # 输出: 3  
8  
9 # 查看字符串长度  
10 print(len(my_string)) # 输出: 14  
11  
12 # 字符串中字符的替换  
13 new_string = my_string.replace('world', 'Python')  
14 print(new_string) # 输出: " hello, Python! "  
15  
16 # 字符串的分割  
17 split_string = my_string.split(',')  
18 print(split_string) # 输出: [' hello', ' world! ']  
19  
20 # 字符串的规整  
21 trimmed_string = my_string.strip()  
22 print(trimmed_string) # 输出: "hello, world!"
```

# 序列类型总结

列表(list), 字符串(str), 元组(tuple) 本质上都是序列 (sequence) , 序列都可以用下标访问



序列的特点如下图所示

| 序列特点      | 举例                                                    |
|-----------|-------------------------------------------------------|
| 内容连续, 有序  | 可以按顺序访问                                               |
| 可以使用下标索引  | <code>sequence[]</code> <code>sequence.index()</code> |
| 可迭代 (可循环) | <code>for item in sequence</code>                     |
| 支持切片      | <code>sequence_new = sequence[:1]</code>              |

序列支持切片操作，即从序列中取出一个子序列，语法如下

▼ 序列切片伪代码

Python |

```
1 子序列 = 序列[start : stop : step]
```

- start: 起始下标索引(取), 如果省略, 默认从序列的开头开始 (索引 0)
- stop: 结束下标索引(不取), 如果省略, 默认到序列的末尾
- step: 步长, 如果省略, 则默认为1; 步长的正负代表取值顺序

课堂口诀: 前取后不取, 步长正负决定取值顺序

## 4. 集合

集合 (set) 用于存储不重复的元素集合。集合是无序的, 因此无法通过索引访问元素

集合三个特点: 无序、元素可变、元素不可重复



## 4.1 集合的常见操作

| 操作         | 方法/属性                                                   | 示例代码                            |
|------------|---------------------------------------------------------|---------------------------------|
| 创建集合       | <code>set()</code> 或 <code>{}</code>                    | <code>my_set = {1, 2, 3}</code> |
| 添加元素       | <code>.add(元素)</code>                                   | <code>my_set.add(4)</code>      |
| 移除元素       | <code>.remove(元素)</code><br>或 <code>.discard(元素)</code> | <code>my_set.remove(2)</code>   |
| 清空集合       | <code>.clear()</code>                                   | <code>my_set.clear()</code>     |
| 获取集合大小     | <code>len(集合)</code>                                    | <code>len(my_set)</code>        |
| 检查元素是否在集合中 | <code>元素 in 集合</code>                                   | <code>3 in my_set</code>        |

## 4.2 集合的运算

集合运算包括并集、交集和差集，这些运算可以使用操作符或集合的方法来实现

| 操作 | 方法                                                           | 示例代码                                                                   |
|----|--------------------------------------------------------------|------------------------------------------------------------------------|
| 并集 | <code>.union(其他集合)</code><br>或 <code> </code> 操作符            | <code>set1.union(set2)</code><br>或 <code>set1   set2</code>            |
| 交集 | <code>.intersection(其他集合)</code><br>或 <code>&amp;</code> 操作符 | <code>set1.intersection(set2)</code><br>或 <code>set1 &amp; set2</code> |
| 差集 | <code>.difference(其他集合)</code><br>或 <code>-</code> 操作符       | <code>set1.difference(set2)</code><br>或 <code>set1 - set2</code>       |

#### 集合运算示例

Python |

```
1 set1 = {1, 2, 3, 4}
2 set2 = {3, 4, 5, 6}
3
4 # 并集
5 union_set = set1 | set2
6 # union_set = set1.union(set2) # 与上一行代码等价
7 print(union_set) # 输出: {1, 2, 3, 4, 5, 6}
8
9 # 交集
10 intersection_set = set1 & set2
11 # intersection_set = set1.intersection(set2) # 与上一行代码等价
12 print(intersection_set) # 输出: {3, 4}
13
14 # 差集
15 difference_set = set1 - set2
16 # difference_set = set1.difference(set2) # 与上一行代码等价
17 print(difference_set) # 输出: {1, 2}
```

## 4.3 集合的遍历

集合无法用 `while` 循环遍历，因为集合是无序的（即不存在下标索引），但可以使用 `for` 循环遍历集合中的每个元素

#### 集合遍历示例

Python |

```
1 my_set = {1, 2, 3, 4, 5}
2
3 # 使用 for 循环遍历集合
4 for element in my_set:
5     print(element)
```

## 5. 字典

Python 的字典（dict）是键值对（key-value pair）的无序集合。字典中的每个键（key）都是唯一的，键和值通过冒号分隔

字典四个特点：元素都是键值对key-value的形式、无序、元素可变、元素（键key）不可重复

## 5.1 字典的定义语法

字典定义示例Python

```
1 # 创建一个空字典
2 empty_dict = {}
3
4 # 创建一个包含键值对的字典
5 my_dict = {
6     "name": "Alice",
7     "age": 25,
8     "city": "New York"
9 }
```

可以通过key访问或修改字典的value，示例如下

字典元素访问与修改示例Python

```
1 # 定义字典
2 my_dict = {"name": "Alice", "age": 25, "city": "New York"}
3
4 # 访问字典中的值
5 print(my_dict["name"]) # 输出: Alice
6
7 # 修改字典中的值
8 my_dict["age"] = 26
9 print(my_dict) # 输出: {'name': 'Alice', 'age': 26, 'city': 'New York'}
```

## 5.2 字典的常用操作总结

| 操作       | 方法/属性                          | 示例代码                             |
|----------|--------------------------------|----------------------------------|
| 获取值      | <code>dict[key]</code>         | <code>my_dict["name"]</code>     |
| 添加/修改键值对 | <code>dict[key] = value</code> | <code>my_dict["age"] = 26</code> |
| 删除键值对    | <code>del dict[key]</code>     | <code>del my_dict["city"]</code> |
| 获取所有键    | <code>.keys()</code>           | <code>my_dict.keys()</code>      |
| 获取所有值    | <code>.values()</code>         | <code>my_dict.values()</code>    |
| 获取所有键值对  | <code>.items()</code>          | <code>my_dict.items()</code>     |

|           |                                    |                                                 |
|-----------|------------------------------------|-------------------------------------------------|
| 检查键是否在字典中 | <code>key in dict</code>           | <code>'name' in my_dict</code>                  |
| 清空字典      | <code>.clear()</code>              | <code>my_dict.clear()</code>                    |
| 获取值（带默认值） | <code>.get(key, default)</code>    | <code>my_dict.get("name", "Unknown")</code>     |
| 删除键值对并返回值 | <code>.pop(key, default)</code>    | <code>my_dict.pop("age", "Not Found")</code>    |
| 更新字典      | <code>.update(another_dict)</code> | <code>my_dict.update({"country": "USA"})</code> |

## 5.3 字典的遍历

使用 `.keys()`、`.values()` 和 `.items()` 可以遍历字典的键、值和键值对。

```

1  my_dict = {"name": "Alice", "age": 25, "city": "New York"}
2
3  # 遍历所有键
4  for key in my_dict.keys():
5      print(key)
6  # 输出:
7  # name
8  # age
9  # city
10
11 # 遍历所有值
12 for value in my_dict.values():
13     print(value)
14 # 输出:
15 # Alice
16 # 25
17 # New York
18
19 # 遍历所有键值对
20 for key, value in my_dict.items():
21     print(f"{key}: {value}")
22 # 输出:
23 # name: Alice
24 # age: 25
25 # city: New York

```

## 数据容器总结

课堂总结表格1如下

| 数据容器        | 元素类型支持 | 下标索引 | 能否重复 | 能否修改 | 是否有序 | 使用场景    |
|-------------|--------|------|------|------|------|---------|
| 列表 list []  | 任意     | 支持下标 | 可重复  | 可修改  | 有序   | 可修改、可重复 |
| 元组 tuple () | 任意     | 支持下标 | 可重复  | —    | 有序   | 不可修改    |
| 字符串 str ""  | 仅字符    | 支持下标 | 可重复  | —    | 有序   | 字符串     |

|            |                                   |   |   |     |   |                 |
|------------|-----------------------------------|---|---|-----|---|-----------------|
| 集合 set {}  | 任意                                | – | – | 可修改 | – | 不可重复            |
| 字典 dict {} | key:value<br>key–除了字典<br>value–任意 | – | – | 可修改 | – | 通过 key 记录 value |

课堂总结表格2如下

| 数据容器        | 循环  | 通用统计函数 | 通用取值函数      | 数据转化    | 通用排序     |
|-------------|-----|--------|-------------|---------|----------|
| 列表 list []  | for | len()  | max() min() | list()  | sorted() |
| 元组 tuple () | for | len()  | max() min() | tuple() | sorted() |
| 字符串 str ""  | for | len()  | max() min() | str()   | sorted() |
| 集合 set {}   | for | len()  | max() min() | set()   | sorted() |
| 字典 dict {}  | for | len()  | max() min() | –       | sorted() |